

AI Assisted Coding

Assignment – 4.2

Name: G.Vamshi

Batch: 21

HtNo: 2303A51448

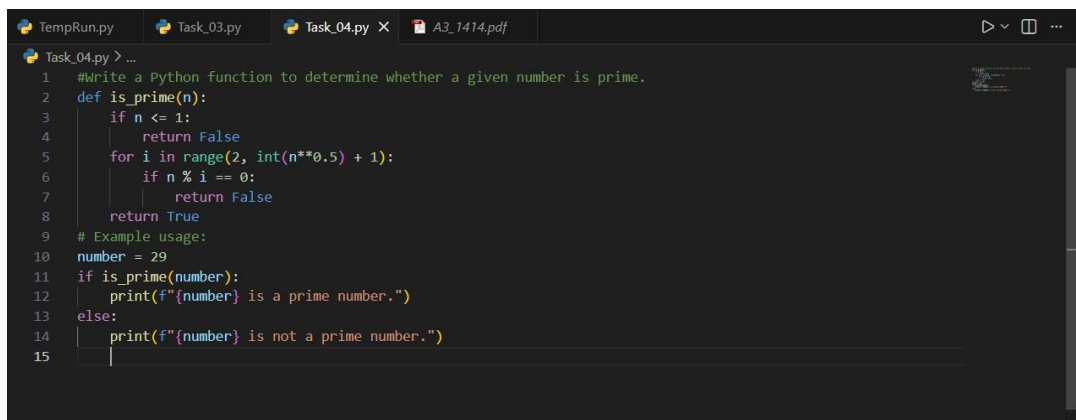
Task Description-1

- Zero-shot: Prompt AI with only the instruction. Write a Python function to determine whether a given number is prime

Expected Output-1

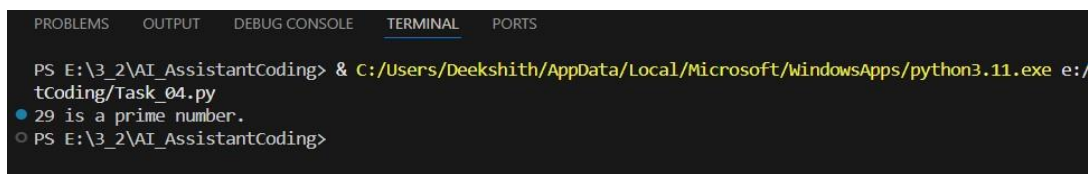
- A basic Python function to check if a number is prime, demonstrating correct logical conditions without relying on examples or additional context.

Code:



```
TempRun.py Task_03.py Task_04.py X A3_1414.pdf
Task_04.py > ...
1 #Write a Python function to determine whether a given number is prime.
2 def is_prime(n):
3     if n <= 1:
4         return False
5     for i in range(2, int(n**0.5) + 1):
6         if n % i == 0:
7             return False
8     return True
9 # Example usage:
10 number = 29
11 if is_prime(number):
12     print(f"{number} is a prime number.")
13 else:
14     print(f"{number} is not a prime number.")
15 |
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/tCoding/Task_04.py
● 29 is a prime number.
○ PS E:\3_2\AI_AssistantCoding>
```

Final Observation:

The `is_prime(n)` function checks whether a number is prime by first making sure the number is greater than 1, since 0, 1, and negative numbers are not prime. It then goes through the

numbers from 2 to $n-1$ to see if any of them divide the given number exactly. If it finds even one such number, it returns False, meaning the number is not prime

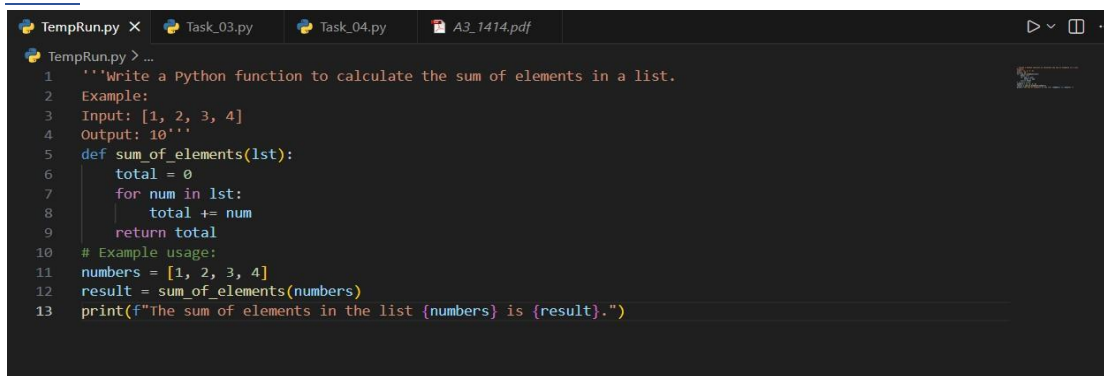
Task Description-2

- One-shot: Provide one example: Input: [1, 2, 3, 4], Output: 10 to help AI generate a function that calculates the sum of elements in a list.

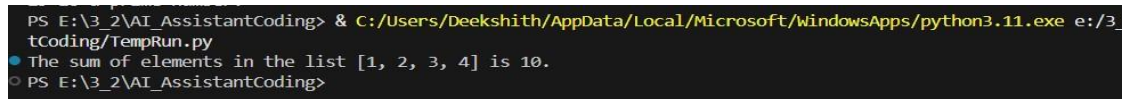
Expected Output-2

- A correct conversion function guided by the single example.

Code:

A screenshot of a Python IDE window titled 'TempRun.py'. The code defines a function 'sum_of_elements' that takes a list 'lst' and returns the sum of its elements. It includes a docstring with an example: 'write a Python function to calculate the sum of elements in a list. Example: Input: [1, 2, 3, 4] Output: 10'. The function uses a loop to calculate the sum. Below the function, there is an example usage: 'numbers = [1, 2, 3, 4]', 'result = sum_of_elements(numbers)', and 'print(f"The sum of elements in the list {numbers} is {result}.")'.

Output:

A screenshot of a terminal window. The command prompt shows the execution of a Python script: 'PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2Coding/TempRun.py'. The output is: 'The sum of elements in the list [1, 2, 3, 4] is 10.'.

Final Observation:

The `sum_of_elements(arr)` function calculates the total sum of all elements in a list by starting with a variable `total` set to zero and then adding each number in the list one by one. After the loop finishes, the final value of `total` is returned as the sum of the list. Since this function was generated using one-shot prompting, where a single input-output example was provided, the example helped guide the AI to clearly understand the task and produce a correct and simple function for summing list elements.

Task Description-3

- Few-shot: Give 2–3 examples to create a function that extracts digits from an alphanumeric string.

Expected Output-3

Zero Shot:

Code:

```
TempRun.py X Task_04.py
TempRun.py > ...
1 #Write a Python function to count the number of vowels in a string.
2 def count_vowels(s):
3     vowels = 'aeiouAEIOU'
4     count = 0
5     for char in s:
6         if char in vowels:
7             count += 1
8     return count
9 # Example usage:
10 input_string = "Hello World"
11 vowel_count = count_vowels(input_string)
12 print(f"The number of vowels in '{input_string}' is {vowel_count}.")
```

Output:

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe tCoding/TempRun.py
• The number of vowels in 'Hello World' is 3.
PS E:\3_2\AI_AssistantCoding>
```

Few Shot:

Code:

```
TempRun.py X Task_04.py
TempRun.py > ...
1 '''Write a Python function to count the number of vowels in a string.
2 Examples:
3 Input: "hello" → Output: 2
4 Input: "ChatGPT" → Output: 2
5 Input: "AEIOU" → Output: 5'''
6 def count_vowels(s):
7     vowels = 'aeiouAEIOU'
8     count = 0
9     for char in s:
10         if char in vowels:
11             count += 1
12     return count
13 # Example usage:
14 input_string = "Deekshith"
15 vowel_count = count_vowels(input_string)
16 print(f"The number of vowels in '{input_string}' is {vowel_count}.")
```

Output:

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/TempRun.py
• The number of vowels in 'Deekshith' is 3.
PS E:\3_2\AI_AssistantCoding>
```

Output Comparison:

Zero-shot version:

The function is clear, simple, and correct, using a loop and counter variable.

Few-shot version:

The function is more refined and compact. The examples helped the model clearly understand that both uppercase and lowercase vowels must be counted, leading to a more confident and optimized implementation.

Final Observation:

In the zero-shot approach, the AI generated a basic vowel-counting function based only on the instruction, which resulted in a straightforward loop-based solution. In the few-shot approach, the given examples clearly showed what should be considered as vowels and how the output should look. These examples helped the AI better understand the pattern and expectations, which led to a cleaner and slightly optimized solution. This comparison shows that providing examples improves clarity and often results in better-structured and more accurate outputs.

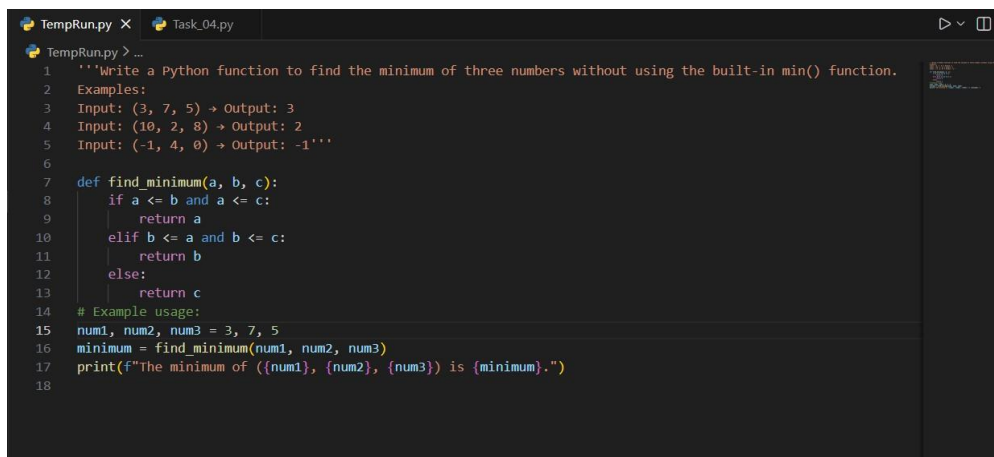
Task Description-5

- Use few-shot prompting with 3 sample inputs to generate a function that determines the minimum of three numbers without using the built-in min() function.

Expected Output-5

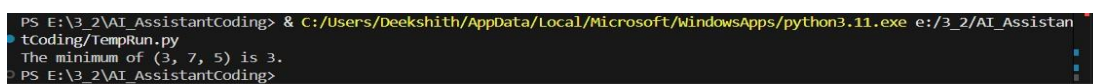
- A function that handles all cases with correct logic based on example patterns.

Code:



```
TempRun.py X Task_04.py
TempRun.py > ...
1 '''Write a Python function to find the minimum of three numbers without using the built-in min() function.
2 Examples:
3 Input: (3, 7, 5) -> Output: 3
4 Input: (10, 2, 8) -> Output: 2
5 Input: (-1, 4, 0) -> Output: -1'''
6
7 def find_minimum(a, b, c):
8     if a <= b and a <= c:
9         return a
10    elif b <= a and b <= c:
11        return b
12    else:
13        return c
14    # Example usage:
15    num1, num2, num3 = 3, 7, 5
16    minimum = find_minimum(num1, num2, num3)
17    print(f"The minimum of ({num1}, {num2}, {num3}) is {minimum}.")
18
```

Output:



```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_Assistan
tCoding/TempRun.py
The minimum of (3, 7, 5) is 3.
PS E:\3_2\AI_AssistantCoding>
```

Final Observation:

The find_minimum(a, b, c) function compares the three given numbers using conditional statements. It first checks whether a is smaller than or equal to both b and c. If not, it then checks whether b is smaller than or equal to the other two. If neither of these conditions is true, the function returns c as the minimum. Since this function was generated using fewshot prompting, the three examples clearly showed the expected pattern of comparing values and returning the smallest one. These examples helped the AI design logic that correctly handles all cases, including negative numbers.

